Discussion 9.1 Peer-to-peer discussion	
Members: Abon, Aldous Amora, Ruzyl Bryan Bonita, Kier Nathaniel Caccam, Neil Alexander	**Date:** Sep. 8, 2026
Course/Section: CPE 014A-CPE31S2	**Instructor:** Engr. Menchie Rosales

Problems Encountered		
Problem	**Cause**	**Solution**
1. Insufficient power and ground connections	Too many components are sharing limited power and GND points.	Use the power and ground bus terminals on the breadboard to distribute the connections.
2. Motors are weak or not moving	The motors are not getting enough power or the wires are loose.	Check the motor wiring and make sure the motor driver is receiving enough voltage.
3. Sensors are not detecting properly	Sensors may be connected to the wrong pins or have unstable power.	Check the sensor wiring and make sure the VCC, GND, and signal pins are connected correctly.
4. ESP32 keeps restarting	The voltage drops when the motors start running.	Use a stable power supply and separate the motor power from the ESP32 power when needed.
5. SumoBot moves in the wrong direction	The motor wires may be connected in the wrong polarity.	Switch the motor wires or adjust the motor direction in the code.
Pre-assumptions		
Problem	**Cause (Pre-Assumption)**	**Solution (Pre-Planned Action)**
6. ESP32 might not read sensor signals correctly or gives weird values	We assume the sensors we bought are 5V type, but the ESP32 runs on 3.3V logic. The signal voltage might be too high for the ESP32 to interpret properly, or the sensor won't detect the ESP32's 3.3V output.	We will check the sensor datasheet first. If they are 5V-only, we will add a simple voltage divider (two resistors) on the signal wire going into the ESP32, or we will buy 3.3V-compatible sensors to avoid this issue entirely.
7. Motor driver chip (like L298N) might overheat and get very hot	We assume the motors will draw high stall current during a push, and the driver has no proper heatsink.	We will immediately unplug power if we feel heat. We'll put a small heatsink on the chip (or tape a coin

	There's also a chance that exposed motor wire terminals accidentally touch each other or the metal chassis, causing a short circuit.	with thermal paste). We'll also wrap electrical tape around exposed motor terminals so they don't short out against anything.
8. Powerbank might keeps shutting down or ESP32 restarts when motors try to spin	We assume the powerbank has overcurrent protection (OCP). When the motors suddenly start, they draw a huge "inrush current" that trips the powerbank's safety, making it cut off power completely. Also, our buck converter might not be rated high enough for the motor's stall current, so it overheats or drops voltage.	We will add a large capacitor (e.g., 1000µF or 2200µF) right at the input of the buck converter to absorb the sudden current spike and prevent the powerbank from tripping. We will also check our buck converter's current rating we assume we need at least 2A-3A continuous for the motors. We will set the buck converter output to the motor's rated voltage (e.g., 6V or 7.4V) and never exceed the motor's max limit.
9. Sensor readings become random or unstable when motors are running	We assume the spinning motors create electrical noise that messes with the sensor's power supply. We also assume we might have connected the sensor GND to a different point than the ESP32 GND, causing a floating ground.	We will add a 100uF capacitor near the sensor's VCC and GND pins to filter out noise. We will also practice star grounding, meaning we will tie all GNDs (ESP32, driver, sensors, battery negative) together at ONE single common point on the breadboard.
10. The bot might move in the opposite direction of what we expect	We assume the motor wires are connected with reversed polarity on one side compared to the other. We also assume that we might try to fix this in code first, only to confuse ourselves later when we swap wires again.	We will do a simple hardware test first, write a short code to spin both motors forward. If one spins backward, we will physically swap the two wires on only that one motor. We will fix the hardware direction first before adjusting the software, so we don't get mixed up.